


## Ampliación de Bases de Datos

Grado en Ingeniería del Software

Curso académico 2026/2027

# TPI 2

## Aula Viva: análisis de ocupación y ruido con Spark SQL

**Estudiante:** Miguel Garcia Galan
**Profesor:** Dr. Anier Soria Lorente
**Fecha:** 27/10/2026
**Repositorio:** https://github.com/miguelitillo14/tpi2-aula-viva

Universidad Loyola Andalucía

# Índice

**1. Problema real y contexto** **2**

**2. Necesidad relacionada con los datos y objetivos** **2**

2.1. Necesidad identificada . . . . . 2

2.2. Objetivo general . . . . . 2

2.3. Objetivos específicos . . . . . 3

**3. Conceptos y tecnologías aplicadas** **3**

3.1. Spark SQL para datos estructurados . . . . . 3

3.2. Integración con el sistema de información empresarial . . . . . 3

3.3. Calidad, trazabilidad y escala . . . . . 3

**4. Desarrollo del trabajo** **4**

4.1. Diseño o planteamiento de la solución . . . . . 4

4.2. Implementación o procedimiento . . . . . 4

**5. Resultados obtenidos** **5**

**6. Análisis e interpretación de los resultados** **6**

**7. Decisiones tomadas y justificación** **6**

**8. Conclusiones y aprendizaje** **7**

**9. Declaración de uso de inteligencia artificial** **7**

**Referencias** **7**

**A. Reproducción del prototipo** **8**

**B. Alcance de la evidencia** **8**

# 1. Problema real y contexto

Aula Viva es la propuesta de aplicación web para consultar espacios de estudio del campus y comunicar observaciones recientes sobre ocupación y ruido. La necesidad parte de una situación concreta: un estudiante que llega al campus no dispone necesariamente de información actualizada para decidir qué espacio está libre y es adecuado para estudiar. La propuesta inicial delimita un campus, informes manuales y consulta de espacios; no contempla por ahora sensores automáticos ni otros campus.

El trabajo aborda una fase analítica de ese problema. Aula Viva produce informes de ocupación y ruido, mientras que el sistema de información institucional mantiene el catálogo autorizado de espacios, edificios, capacidades y estado de actividad. Consultar cada fuente por separado dificulta ofrecer un estado contextualizado: un informe reciente debe vincularse con el espacio correcto y su capacidad para poder interpretarse.

No se dispone de acceso a un sistema empresarial real ni de informes reales del campus. Por ello, el prototipo utiliza dos CSV sintéticos, identificados expresamente como datos de demostración. El problema y la arquitectura son los de Aula Viva; las cifras que aparecen en la memoria no se presentan como mediciones del campus.

# 2. Necesidad relacionada con los datos y objetivos

## 2.1. Necesidad identificada

Se necesita integrar datos operativos heterogéneos: el catálogo relativamente estable del campus y los informes que cambian a lo largo del día. El procesamiento debe validar identificadores y rangos, resolver varios informes del mismo espacio y obtener un estado reciente listo para consulta desde la aplicación. Spark SQL permite expresar limpieza, unión y selección de registros mediante operaciones sobre datos estructurados. La integración con el sistema institucional es el límite entre esos datos de negocio y la analítica de Aula Viva.

## 2.2. Objetivo general

Diseñar e implementar un prototipo reproducible con Spark SQL que combine un catálogo de espacios y observaciones de ocupación y ruido, genere el último estado válido por espacio y un resumen por edificio, y defina una vía segura de integración con el sistema de información del campus.

### 2.3. Objetivos específicos

- Representar como fuentes separadas el catálogo institucional y los informes de Aula Viva.
- Aplicar validaciones de clave, fecha y rangos antes de agregar los informes.
- Usar una consulta SQL con ventana para seleccionar el informe más reciente de cada espacio.
- Obtener un resumen por edificio y definir cómo consumirlo desde la aplicación.
- Delimitar qué se ha ejecutado, qué es una salida esperada y qué requeriría acceso a sistemas reales.

## 3. Conceptos y tecnologías aplicadas

### 3.1. Spark SQL para datos estructurados

Spark SQL es el componente de Apache Spark para procesar datos estructurados mediante SQL y DataFrames. En este caso, el catálogo y los informes se registran como vistas temporales y una consulta SQL los limpia, combina y ordena. El propósito no es afirmar que un conjunto pequeño necesite un clúster, sino aplicar el modelo de procesamiento distribuible a una tarea que podría crecer cuando se reciban observaciones de múltiples espacios y periodos.

### 3.2. Integración con el sistema de información empresarial

El sistema institucional es la fuente maestra de los espacios y sus capacidades; Aula Viva es la fuente de informes comunitarios. La integración propuesta es de lectura: Spark consume una exportación controlada o una réplica mediante JDBC, combina las fuentes y publica una salida analítica para la aplicación. No se escriben observaciones en el sistema administrativo ni se reemplaza su catálogo. Como no se proporcionaron conexión, credenciales ni autorización, el prototipo simula el origen JDBC con el CSV `spaces_sample.csv`.

### 3.3. Calidad, trazabilidad y escala

La clave `space_id` relaciona las dos fuentes. El `INNER JOIN` descarta observaciones sin espacio correspondiente; las condiciones de validación rechazan fechas ilegibles, ocupaciones fuera de $[0, 1]$ y ruido fuera del intervalo definido para el prototipo. Una función

de ventana elige un único informe vigente por espacio. La misma lógica deja preparada una salida separada para el resumen, sin alterar el dato de origen.

# 4. Desarrollo del trabajo

## 4.1. Diseño o planteamiento de la solución

El flujo tiene cuatro etapas: (1) leer un extracto del catálogo institucional y los informes de Aula Viva; (2) validarlos y relacionarlos por identificador; (3) seleccionar el último informe válido de cada espacio y calcular el resumen por edificio; y (4) publicar CSV de salida para que un servicio de lectura de la aplicación los consuma. La primera versión corre en modo local con Spark; el modo de ejecución podría cambiarse a un clúster sin modificar la consulta de negocio.

| Fuente o componente | Datos | Responsabilidad en el flujo |
|---|---|---|
| Sistema institucional | Identificador, edificio, nombre, capacidad y estado | Catálogo maestro de espacios; solo lectura |
| Aula Viva | Identificador del informe, espacio, fecha, ocupación y ruido | Observaciones comunitarias con marca temporal |
| Spark SQL | Vistas estructuradas y consulta con unión y ventana | Validación, integración y agregación |
| Aplicación Aula Viva | CSV analítico de salida en el prototipo | Mostrar estado reciente sin consultar cada origen por separado |

## 4.2. Implementación o procedimiento

El proyecto contiene `scripts/run_spark_sql.py`, dos CSV sintéticos y los comandos de instalación en `README.md`. El script crea una sesión Spark local, carga los ficheros como vistas SQL, descarta filas que no pueden vincularse con el catálogo o no superan las validaciones y conserva el último informe por espacio. Después calcula la ocupación media del último estado por edificio con Spark SQL. Dado que la muestra es pequeña, el script recoge estas tablas y Python las guarda en dos CSV locales; así se evita exigir `winutils.exe` en Windows. Para datos de gran volumen, la salida debe dirigirse a almacenamiento distribuido o a un sistema de información mediante un conector adecuado.

La consulta principal usa `ROW_NUMBER() OVER (PARTITION BY space_id ORDER BY instante DESC)`. El orden secundario por identificador del informe hace determinista la selección cuando dos informes comparten marca temporal. Para conectar un sistema compatible real, se sustituiría la lectura de `spaces_sample.csv` por el conector JDBC

de Spark, con el controlador apropiado y secretos fuera del código. La salida de esta prueba no se escribe en el sistema institucional.

```
WITH validos AS (
  SELECT r.report_id, r.space_id,
         to_timestamp(r.reported_at, 'yyyy-MM-dd HH:mm:ss') AS instante,
         CAST(r.occupancy_ratio AS DOUBLE) AS ocupacion,
         CAST(r.noise_db AS DOUBLE) AS ruido_db,
         s.building, s.space_name, CAST(s.capacity AS INT) AS capacidad,
         ROW_NUMBER() OVER (
           PARTITION BY r.space_id
           ORDER BY to_timestamp(r.reported_at, 'yyyy-MM-dd HH:mm:ss') DESC,
                    r.report_id DESC
         ) AS orden
  FROM informes_brutos r
  INNER JOIN catalogo_espacios s ON r.space_id = s.space_id
  WHERE s.active = 'true'
    AND to_timestamp(r.reported_at, 'yyyy-MM-dd HH:mm:ss') IS NOT NULL
    AND CAST(r.occupancy_ratio AS DOUBLE) BETWEEN 0.0 AND 1.0
    AND CAST(r.noise_db AS DOUBLE) BETWEEN 0.0 AND 120.0
)
SELECT report_id, space_id, instante, ocupacion, ruido_db,
       building, space_name, capacidad
FROM validos WHERE orden = 1;
```

Listing 1: Selección del último informe válido y unión con el catálogo

## 5. Resultados obtenidos

Los ficheros de demostración contienen cuatro espacios y siete informes. Seis informes usan un identificador presente en el catálogo y uno (`R007`, espacio `X999`) no tiene correspondencia. Por construcción, la unión interna conserva seis informes antes de elegir el más reciente y la salida final contiene cuatro espacios. La tabla muestra los estados obtenidos al ejecutar el prototipo con Spark SQL.

| Espacio | Último informe de muestra | Ocupación | Edificio |
|---|---|---|---|
| A101 | R002, 09:10 | 0,80 | A |
| A102 | R003, 09:05 | 0,20 | A |
| B201 | R006, 09:18 | 0,50 | B |
| C101 | R005, 09:15 | 0,90 | C |

La ejecución local de Spark SQL terminó correctamente y produjo los cuatro estados mostrados y los resúmenes: edificio A, media 0,50 con dos espacios; edificio B, media 0,50 con un espacio; edificio C, media 0,90 con un espacio. Se comprobaron los ficheros `salida/estado_espacios.csv` y `salida/resumen_edificios.csv`; contienen cuatro filas de estado y tres filas de resumen, respectivamente. El informe de espacio inexistente quedó excluido por la unión con el catálogo. Los avisos de Hadoop relativos a `winutils.exe` y a la biblioteca nativa aparecieron como advertencias, pero no impidieron ejecutar la consulta ni guardar los resultados.

# 6. Análisis e interpretación de los resultados

La salida de referencia muestra cómo la integración añade contexto al informe: el último nivel de ocupación se puede asociar con edificio, nombre y capacidad del espacio. La observación R007 se excluye porque su clave no aparece en el catálogo, evitando que un espacio desconocido entre en el resumen. Para A101, R002 sustituye al informe anterior R001 por ser más reciente. El promedio por edificio describe únicamente los cuatro últimos informes sintéticos del ejemplo; no es un indicador representativo de ocupación real.

El prototipo no demuestra rendimiento ni escalabilidad: los siete registros son intencionadamente pequeños y adecuados para revisar la lógica. Tampoco resuelve informes caducados, usuarios malintencionados, valores sesgados, conflictos simultáneos o una política institucional de anonimización. Antes de mostrar datos a estudiantes habría que definir antigüedad máxima, controles de frecuencia, consentimiento, granularidad y mensaje de incertidumbre.

# 7. Decisiones tomadas y justificación

- **Spark SQL y no un almacén nuevo:** el objetivo del TPI 2 es aplicar el procesamiento estructurado distribuible sobre dos fuentes; Spark se usa como motor de integración analítica.
- **Lectura del sistema institucional:** el catálogo mantiene su autoridad y se evita que un proceso experimental altere información operativa.

- **CSV para la prueba local:** no hay acceso autorizado a la base empresarial. El CSV permite reproducir el mismo contrato de datos sin fingir una conexión real.
- **Identificador estable y ventana:** se relacionan registros por clave, no por nombre; la ventana conserva un solo estado por espacio y una marca temporal.
- **Ocupación como proporción:** 0 significa vacío y 1 capacidad completa, por lo que el rango admite validación simple y facilita comparar espacios de distinto tamaño. El prototipo no estima ocupación a partir de sensores.
- **No usar cluster para la muestra:** un modo local reduce requisitos y facilita reproducir la lógica. La escala no se puede justificar sin volumen real ni medidas de rendimiento.

## 8. Conclusiones y aprendizaje

El TPI aplica Spark SQL a una necesidad de Aula Viva: combinar observaciones cambiantes con el catálogo institucional de espacios para obtener un estado reciente y contextualizado. La propuesta de integración separa el sistema maestro del procesamiento analítico, conserva el origen como solo lectura y explicita qué datos se validan.

La ejecución local confirmó que Spark SQL valida y cruza las dos fuentes, selecciona el último informe de cada espacio y genera los dos CSV de salida. El ejemplo demuestra el funcionamiento del flujo, pero usa datos sintéticos: no mide la ocupación real del campus ni permite evaluar rendimiento a gran escala. La conexión con el sistema institucional real queda pendiente de acceso y autorización.

## 9. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como apoyo para organizar la memoria, revisar la relación con las orientaciones del TPI 2 y preparar el prototipo PySpark y los datos sintéticos de demostración. El estudiante ejecutó el procedimiento localmente con PySpark y Java y contrastó la salida con las tablas de referencia y los CSV generados. No se han utilizado datos personales ni credenciales reales.

## Referencias

[1] Soria Lorente, A., *Trabajos Prácticos Individuales: Orientaciones para el alumnado, Ampliación de Bases de Datos, curso 2026/2027*, Universidad Loyola Andalucía, apartado 6: TPI 2, Tema 5.

[2] Garcia Galan, M., *Aula Viva: propuesta inicial del proyecto*, Universidad Loyola Andalucía, 2026. Documento local del proyecto de asignatura.

[3] Apache Software Foundation, *Spark SQL, DataFrames and Datasets Guide*. https://spark.apache.org/docs/latest/sql-programming-guide.html

[4] Apache Software Foundation, *JDBC To Other Databases*. https://spark.apache.org/docs/latest/sql-data-sources-jdbc.html

[5] Apache Software Foundation, *Spark 3.5.1 Documentation*. https://spark.apache.org/docs/3.5.1/

# A. Reproducción del prototipo

Desde PowerShell, situado en la carpeta del proyecto, instalar las dependencias y ejecutar:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe scripts/run_spark_sql.py --data-dir
   data --output-dir salida
```

Listing 2: Preparación y ejecución local

El entorno requiere Python y una versión compatible de Java. La salida consta de los ficheros `salida/estado_espacios.csv` y `salida/resumen_edificios.csv`. En una integración real, se sustituye el catálogo CSV por una fuente JDBC autorizada y se configuran las credenciales mediante un gestor de secretos; no se guardan contraseñas en el repositorio.

# B. Alcance de la evidencia

Los CSV de entrada son sintéticos. La consulta Spark SQL y la escritura de los ficheros de salida sí se ejecutaron localmente y se verificó su contenido. No se probó un clúster ni hubo conexión al sistema empresarial del campus, por lo que los resultados no son mediciones reales de ocupación.